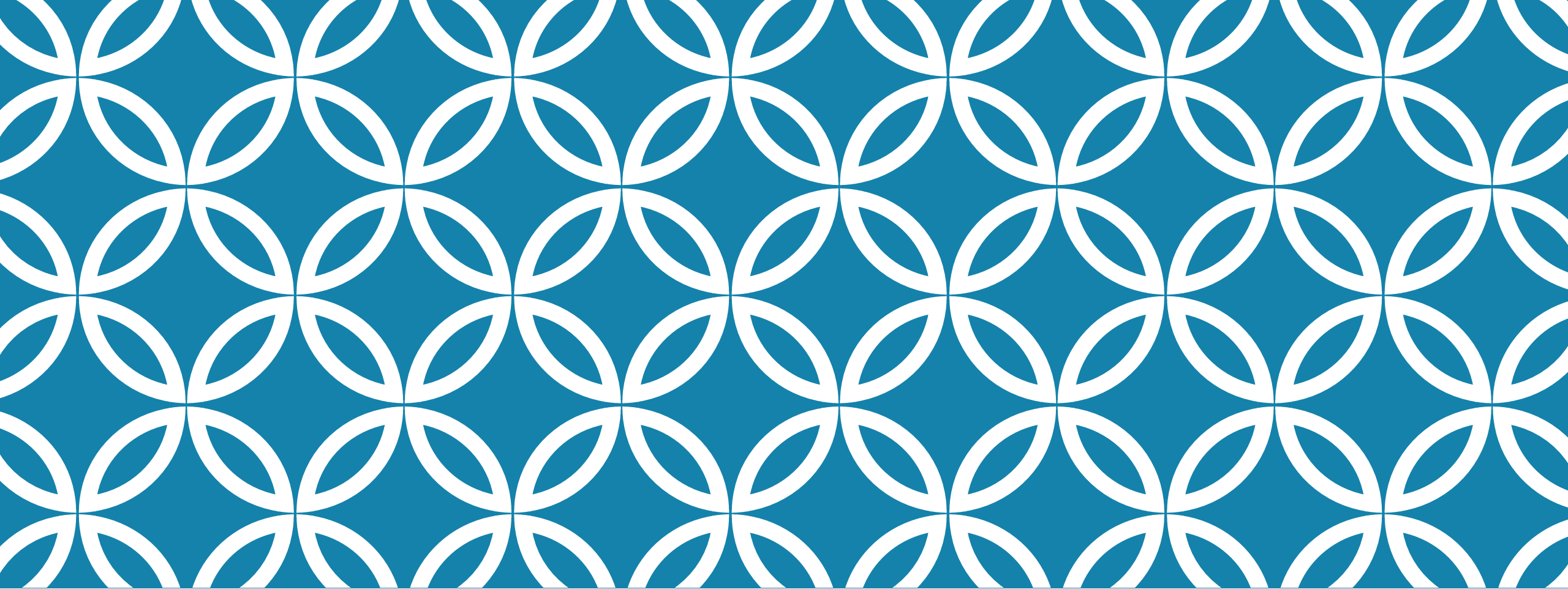


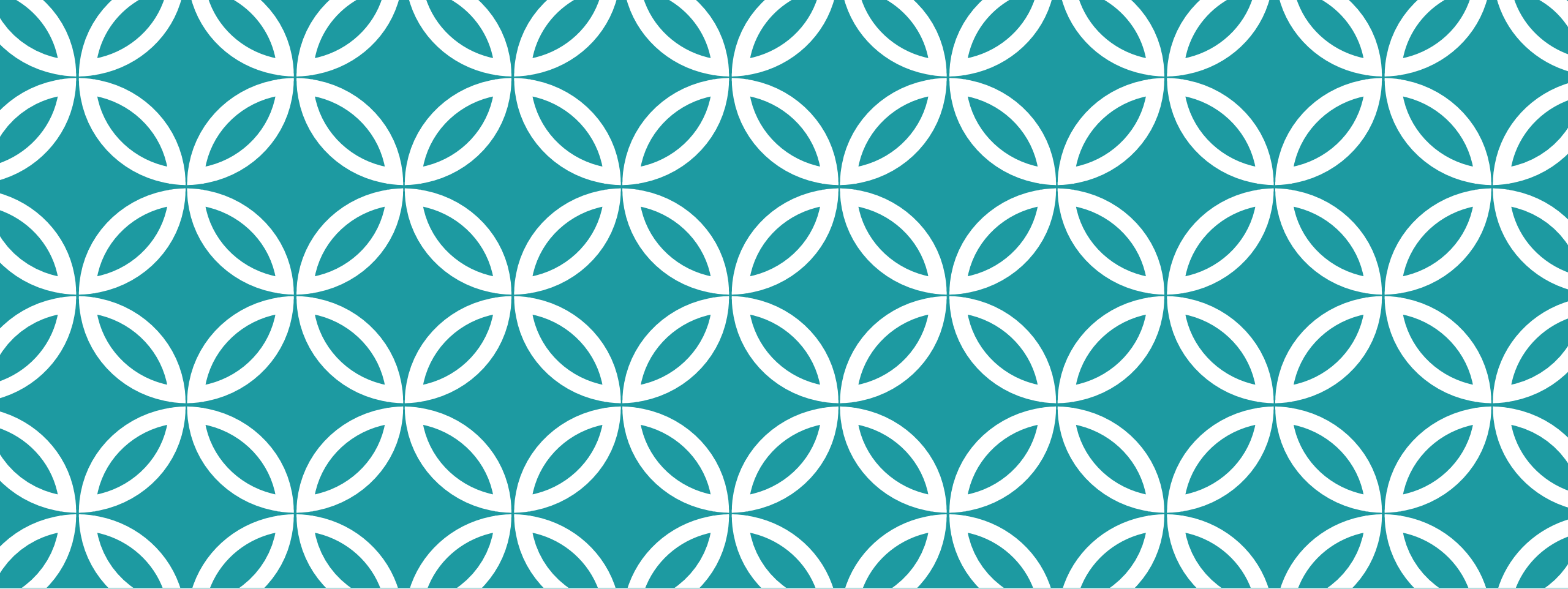


# PARALLEL PROGRAMMING PLATFORMS



# INTRODUCTION

- In a sequential computer the logical view consists of three components namely **processor, memory and datapath**.
- All of these can create a bottleneck with respect to overall processing rate of a computer system.
- Solution? **Multiplicity.... Of datapaths, processing and memory units.**
- In this session we will be talking about serial and implicitly parallel architectures, to figure out that it is possible to re-engineer codes to achieve speed up.



# IMPLICIT PARALLELISM

Section 2.1

# TRENDS IN MICROPROCESSOR ARCHITECTURE

- Microprocessor technologies have had significant improvements in clock speed but produced other performance bottlenecks.
- To resolve this bottlenecks, alternate solutions were proposed, we will enlist some of them in this section
- Lets revisit our major problem... processors clock rates improved. A LOT. Memory technologies didn't.
- Similarly, higher levels of device integrations resulted in large transistor counts. HOW TO BEST USE THEM?
- Solution... execute multiple instructions in a single clock cycle

# PIPELINING & SUPERSCALAR EXECUTIONS

- Overlapping various instructions in instruction execution (fetch, schedule, decode, operand fetch, execute, store). This enables faster execution. We'll see how.
- Consider the assembly line analogy to understand pipelining.
- Assembly of car takes 100 time units, but this can be broken down into 10 pipelined stages of 10 units each... a single assembly line can now produce a car every 10 time units. 10 fold speedup
- With respect to processors, breaking problems into smaller sub-problems enable faster clock rates, as the problem is now smaller.

# PIPELINING & SUPERSCALAR EXECUTIONS

- Note: speed of a single pipeline is limited by the largest atomic task in pipeline.
- Also, typically every fifth to sixth statement is a branch instruction.
- Long instruction pipelines need effective techniques for predicting branch destinations so that pipelining can be effective.
- Also, when pipelines become deeper, the penalty of misprediction increase as more instructions need to be flushed.
- Solution can be using multiple pipelines

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- Consider a processor with two pipelines and the ability to issue two instructions simultaneously.
- Such processors are called super-pipelined processors.
- **Superscalar execution: The ability of a processor to issue multiple instructions in the same cycle.**
- The architecture we are referring to in this example can then be referred to as two-way superscalar or dual issue execution.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

```
1. load R1, @1000
2. load R2, @1008
3. add R1, @1004
4. add R2, @100C
5. add R1, R2
6. store R1, @2000
```

(i)

```
1. load R1, @1000
2. add R1, @1004
3. add R1, @1008
4. add R1, @100C
5. store R1, @2000
```

(ii)

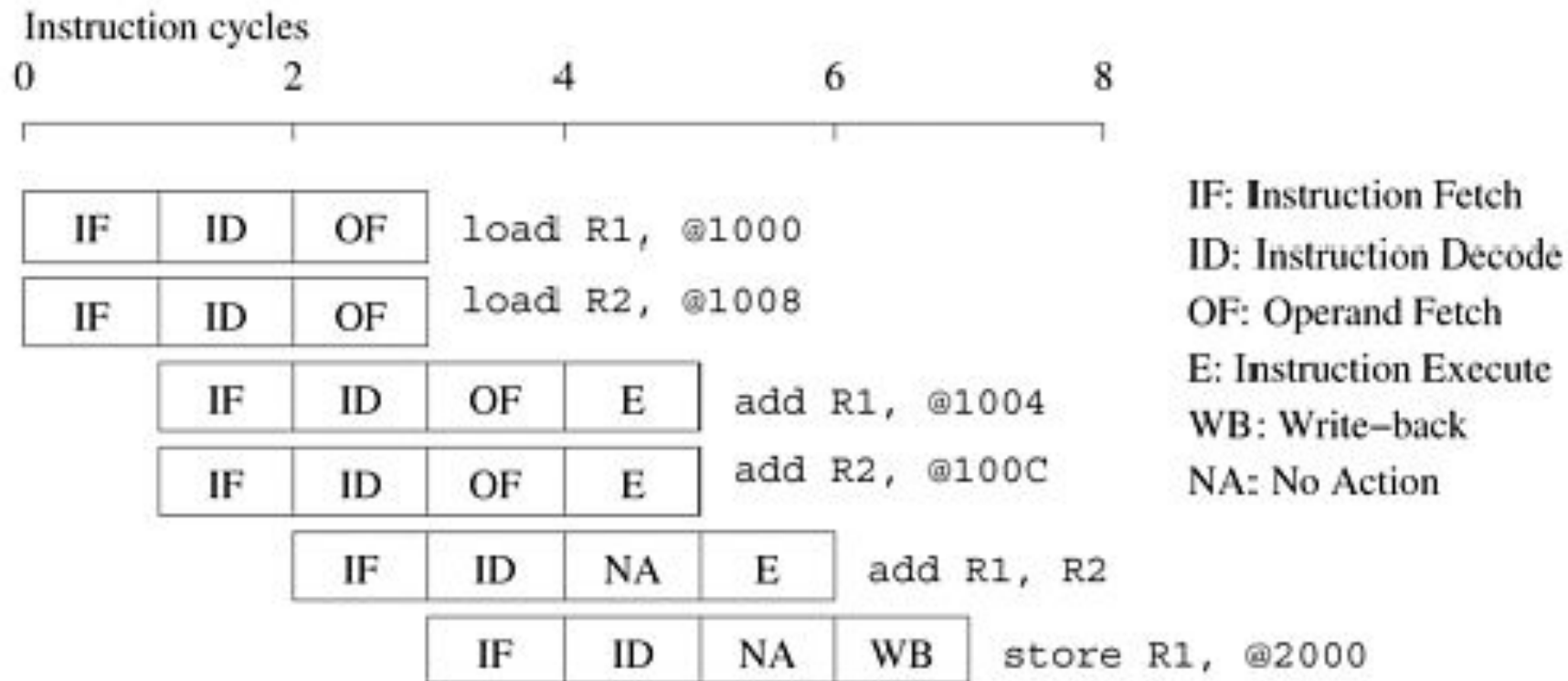
```
1. load R1, @1000
2. add R1, @1004
3. load R2, @1008
4. add R2, @100C
5. add R1, R2
6. store R1, @2000
```

(iii)

(a) Three different code fragments for adding a list of four numbers.



# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE



(b) Execution schedule for code fragment (i) above.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- Consider the execution of the first code fragment in [Figure 2.1](#) for adding four numbers.
- The first and second instructions are independent and therefore can be issued concurrently.
- This is illustrated in the simultaneous issue of the instructions `load R1, @1000` and `load R2, @1008` at  $t = 0$ .
- The instructions are fetched, decoded, and the operands are fetched.
- The next two instructions, `add R1, @1004` and `add R2, @100C` are also mutually independent, although they must be executed after the first two instructions.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- Consequently, they can be issued concurrently at  $t = 1$  since the processors are pipelined. These instructions terminate at  $t = 5$ .
- The next two instructions, `add R1, R2` and `store R1, @2000` cannot be executed concurrently since the result of the former (contents of register `R1`) is used by the latter.
- Therefore, only the `add` instruction is issued at  $t = 2$  and the `store` instruction at  $t = 3$ .
- Note that the instruction `add R1, R2` can be executed only after the previous two instructions have been executed.
- The instruction schedule is illustrated in [Figure 2.1\(b\)](#).
- The schedule assumes that each memory access takes a single cycle. In reality, this may not be the case.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- In principle, superscalar execution seems natural, even simple.
- However, a number of issues need to be resolved.
- First, as illustrated in [Example 2.1](#), instructions in a program may be related to each other.
- The results of an instruction may be required for subsequent instructions.
- **This is referred to as *true data dependency*.**
- For instance, consider the second code fragment in [Figure 2.1](#) for adding four numbers. There is a true data dependency between `load R1, @1000` and `add R1, @1004`, and similarly between subsequent instructions.
- Dependencies of this type must be resolved before simultaneous issue of instructions.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- This has two implications.
- **First, since the resolution is done at runtime, it must be supported in hardware. The complexity of this hardware can be high.**
- Second, the amount of instruction level parallelism in a program is often limited and is a function of coding technique.
- In the second code fragment, there can be no simultaneous issue, leading to poor resource utilization.
- The three code fragments in [Figure 2.1\(a\)](#) also illustrate that in many cases it is possible to extract more parallelism by reordering the instructions and by altering the code.
- Notice that in this example the code reorganization corresponds to exposing parallelism in a form that can be used by the instruction issue mechanism.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- Another source of dependency between instructions results from the finite resources shared by various pipelines.
- As an example, consider the co-scheduling of two floating point operations on a dual issue machine with a single floating point unit.
- Although there might be no data dependencies between the instructions, they cannot be scheduled together since both need the floating point unit.
- This form of dependency in which two instructions compete for a single processor resource is referred to as **resource dependency**.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- The flow of control through a program enforces a third form of dependency between instructions.
- Consider the execution of a conditional branch instruction.
- Since the branch destination is known only at the point of execution, scheduling instructions *a priori* across branches may lead to errors.
- These dependencies are referred to as **branch dependencies** or **procedural dependencies** and are typically handled by speculatively scheduling across branches and rolling back in case of errors.
- Studies of typical traces have shown that on average, a branch instruction is encountered between every five to six instructions.
- Therefore, just as in populating instruction pipelines, accurate branch prediction is critical for efficient superscalar execution.

# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- The ability of a processor to detect and schedule concurrent instructions is critical to superscalar performance.
- For instance, consider the third code fragment in [Figure 2.1](#) which also computes the sum of four numbers.
- The reader will note that this is merely a semantically equivalent reordering of the first code fragment.
- However, in this case, there is a data dependency between the first two instructions – `load R1, @1000` and `add R1, @1004`.
- Therefore, these instructions cannot be issued simultaneously.
- However, if the processor had the ability to look ahead, it would realize that it is possible to schedule the third instruction – `load R2, @1008` – with the first instruction.
- In the next issue cycle, instructions two and four can be scheduled, and so on.



# PIPELINING & SUPERSCALAR EXECUTIONS- EXAMPLE

- In this way, the same execution schedule can be derived for the first and third code fragments.
- However, the processor needs the ability to issue instructions **out-of-order** to accomplish desired reordering.
- The parallelism available in **in-order** issue of instructions can be highly limited as illustrated by this example.
- Most current microprocessors are capable of out - of- order issue and completion.
- This model, also referred to as **dynamic instruction issue**, exploits maximum instruction level parallelism.
- The processor uses a window of instructions from which it selects instructions for simultaneous issue.
- This window corresponds to the lookahead of the scheduler.

# TAKEAWAY

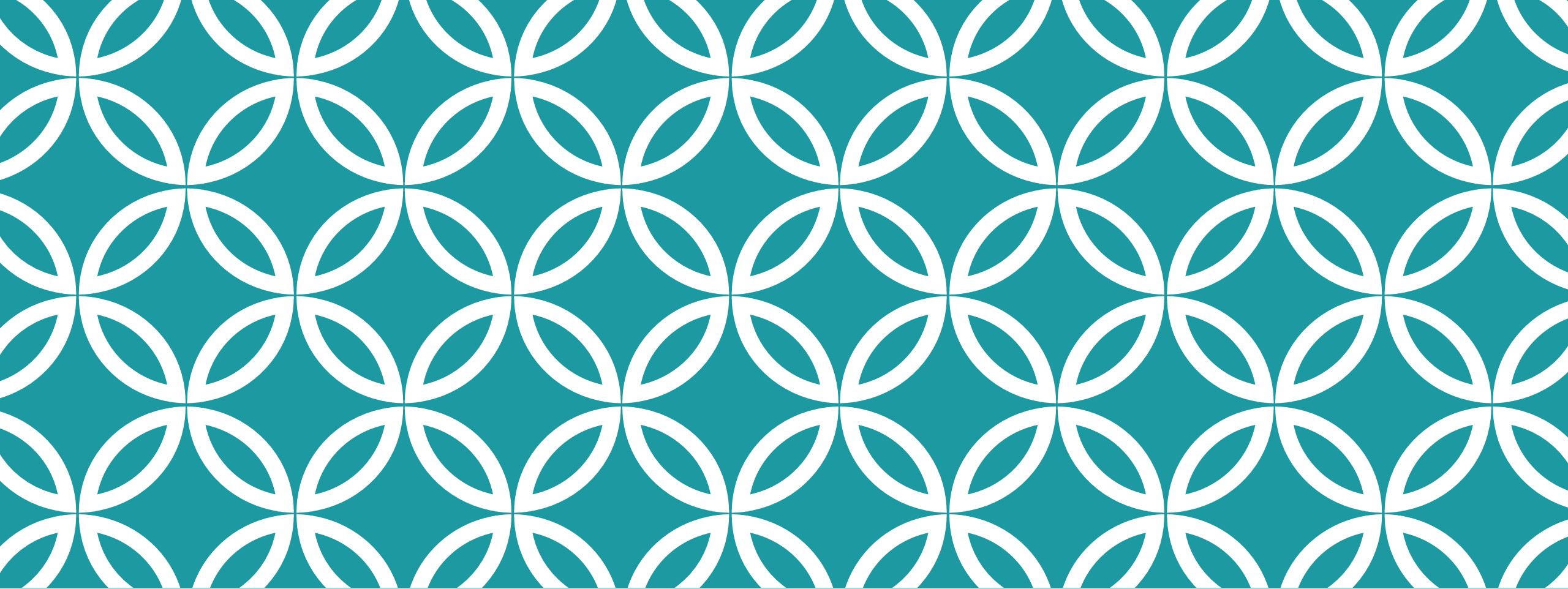
Read horizontal waste and vertical waste, write a brief report (1 page) and submit in next class.

## VERY LONG INSTRUCTION WORD PROCESSORS

- The parallelism extracted by superscalar processors is often limited by the instruction lookahead.
- The hardware logic for dynamic dependency analysis is typically in the range of 5-10% of the total logic on conventional microprocessors (about 5% on the four-way superscalar Sun UltraSPARC).
- This complexity grows roughly quadratically with the number of issues and can become a bottleneck.
- An alternate concept for exploiting instruction-level parallelism used in very long instruction word (VLIW) processors relies on the compiler to resolve dependencies and resource availability at compile time.
- Instructions that can be executed concurrently are packed into groups and parceled off to the processor as a single long instruction word (thus the name) to be executed on multiple functional units at the same time.

## VERY LONG INSTRUCTION WORD PROCESSORS

- Finally, the performance of VLIW processors is very sensitive to the compilers' ability to detect data and resource dependencies and read and write hazards, and to schedule instructions for maximum parallelism.
- Loop unrolling, branch prediction and speculative execution all play important roles in the performance of VLIW processors.
- While superscalar and VLIW processors have been successful in exploiting implicit parallelism, they are generally limited to smaller scales of concurrency in the range of four- to eight-way parallelism.



## **LIMITATIONS OF MEMORY SYSTEM PERFORMANCE**

Section 2.2

# LIMITATIONS OF MEMORY SYSTEM PERFORMANCE

- The effective performance of a program on a computer relies not just on the speed of the processor but also on the ability of the memory system to feed data to the processor.
- At the logical level, a memory system, possibly consisting of multiple levels of caches, takes in a request for a memory word and returns a block of data of size  $b$  containing the requested word after  $l$  nanoseconds.
- Here,  $l$  is referred to as the **latency** of the memory.
- The rate at which data can be pumped from the memory to the processor determines the **bandwidth** of the memory system.

# LIMITATIONS OF MEMORY SYSTEM PERFORMANCE

- As an analogy, if water comes out of the end of a fire hose 2 seconds after a hydrant is turned on, then the latency of the system is 2 seconds.
- Once the flow starts, if the hose pumps water at 1 gallon/second then the 'bandwidth' of the hose is 1 gallon/second.
- **If we need to put out a fire immediately, we might desire a lower latency.**
- This would typically require higher water pressure from the hydrant.
- On the other hand, if we wish to fight bigger fires, we might desire a higher flow rate, necessitating a wider hose and hydrant.
- **Latency and bandwidth both play critical roles in determining memory system performance.**

# IMPROVING EFFECTIVE MEMORY LATENCY USING CACHES

- Handling the mismatch in processor and DRAM speeds has motivated several architectural innovations in memory system design.
- **One such innovation addresses the speed mismatch by placing a smaller and faster memory between the processor and the DRAM.**
- This memory, referred to as the cache, acts as a **low-latency high-bandwidth** storage.
- The data needed by the processor is first fetched into the cache.
- All subsequent accesses to data items residing in the cache are serviced by the cache.
- Thus, in principle, if a piece of data is repeatedly used, the effective latency of this memory system can be reduced by the cache.
- The fraction of data references satisfied by the cache is called the cache ***hit ratio*** of the computation on the system.
- The effective computation rate of many applications is bounded not by the processing rate of the CPU, but by the rate at which data can be pumped into the CPU.



# IMPACT OF MEMORY BANDWIDTH

- Memory bandwidth refers to the rate at which data can be moved between the processor and memory.
- It is determined by the bandwidth of the memory bus as well as the memory units.
- **One commonly used technique to improve memory bandwidth is to increase the size of the memory blocks.**

# IMPACT OF MEMORY BANDWIDTH

- The data layouts were assumed to be such that consecutive data words in memory were used by successive instructions.
- In other words, if we take a computation-centric view, there is a ***spatial locality*** of memory access.
- **If we take a data-layout centric point of view, the computation is ordered so that successive computations require contiguous data.**
- If the computation (or access pattern) does not have spatial locality, then effective bandwidth can be much smaller than the peak bandwidth.

## ALTERNATE APPROACHES FOR HIDING MEMORY LATENCY

- Imagine sitting at your computer browsing the web during peak network traffic hours.
- The lack of response from your browser can be alleviated using one of three simple approaches:
  - (i) we anticipate which pages we are going to browse ahead of time and issue requests for them in advance;
  - (ii) we open multiple browsers and access different pages in each browser, thus while we are waiting for one page to load, we could be reading others; or
  - (iii) we access a whole bunch of pages in one go – amortizing the latency across various accesses.
- The first approach is called ***prefetching***, the second ***multithreading***, and the third one corresponds to spatial locality in accessing memory words

# MULTITHREADING FOR LATENCY HIDING

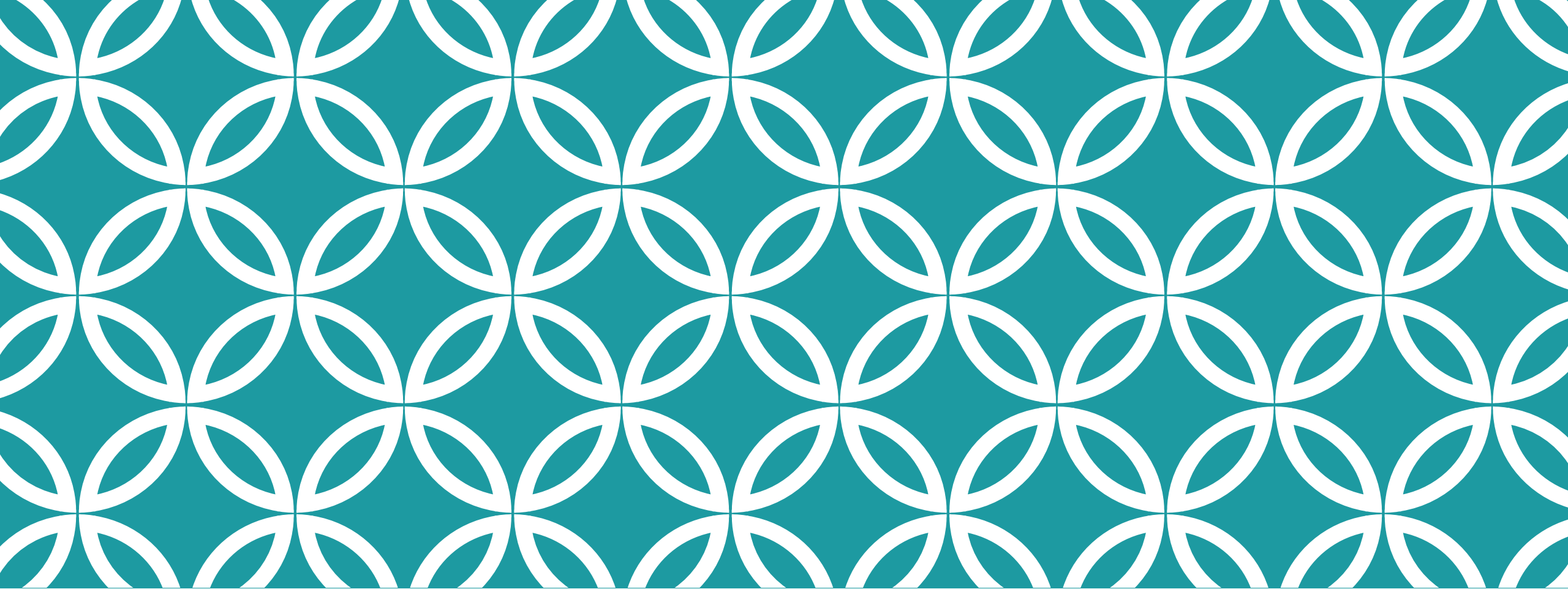
```
for (i=0; i<n; i++)  
    c[i] = dot_product(get_row(a, i),  
                       b);
```

```
for (i=0; i<n; i++)  
    c[i] = create_thread(dot_product,  
                        get_row(a, i), b);
```

This code computes each element of `c` as the dot product of the corresponding row of `a` with the vector `b`. Notice that each dot-product is independent of the other, and therefore represents a concurrent unit of execution

## PREFETCHING FOR LATENCY HIDING

- In a typical program, a data item is loaded and used by a processor in a small time window.
- **If the load results in a cache miss, then the use stalls.**
- A simple solution to this problem is to advance the load operation so that even if there is a cache miss, the data is likely to have arrived by the time it is used.
- However, if the data item has been overwritten between load and use, a fresh load is issued.
- Note that this is no worse than the situation in which the load had not been advanced.
- A careful examination of this technique reveals that prefetching works for much the same reason as multithreading.
- In advancing the loads, we are trying to identify independent threads of execution that have no resource dependency (i.e., use the same registers) with respect to other threads.
- Many compilers aggressively try to advance loads to mask memory system latency.



## DICHOTOMY OF PARALLEL COMPUTING PLATFORMS

Section 2.3

# DICHOTOMY OF PARALLEL COMPUTING PLATFORMS

- Parallel tasks can be specified at various levels of granularity.
- At one extreme, each program in a set of programs can be viewed as one parallel task.
- At the other extreme, individual instructions within a program can be viewed as parallel tasks.
- Between these extremes lie a range of models for specifying the control structure of programs and the corresponding architectural support for them.

# DICHOTOMY OF PARALLEL COMPUTING PLATFORMS

- Processing units in parallel computers either operate under the centralized control of a single control unit or work independently.
- In architectures referred to as ***single instruction stream, multiple data stream*** (SIMD), a single control unit dispatches instructions to each processing unit.
- In an SIMD parallel computer, the same instruction is executed synchronously by all processing units



## SIMD EXAMPLE

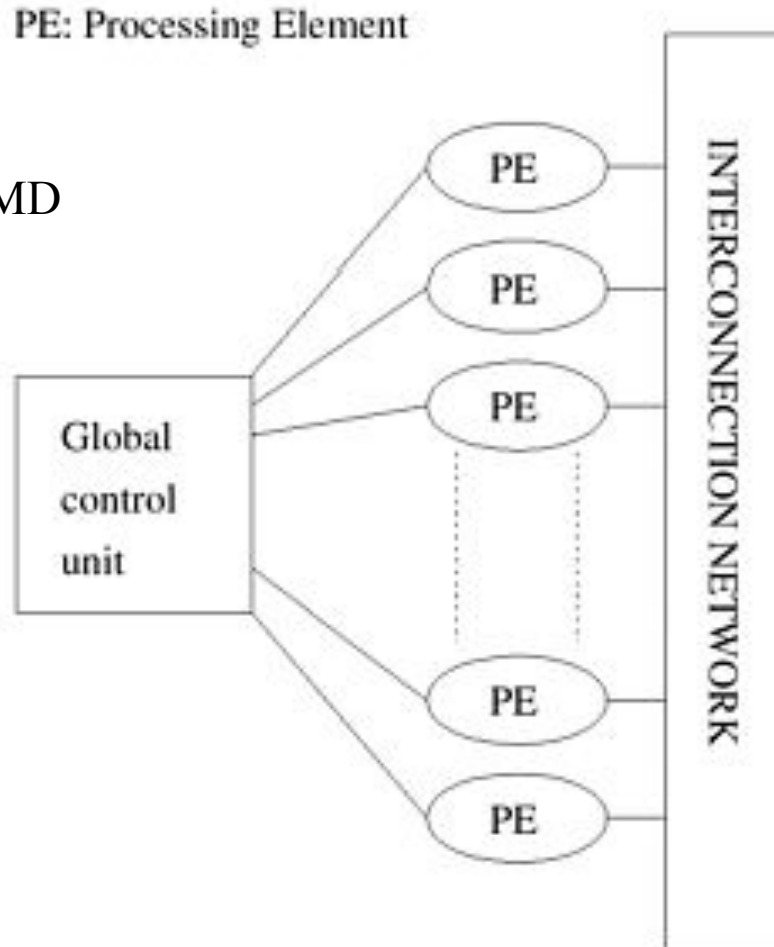
```
for (i = 0; i < 1000; i++)
```

```
c[i] = a[i] + b[i];
```

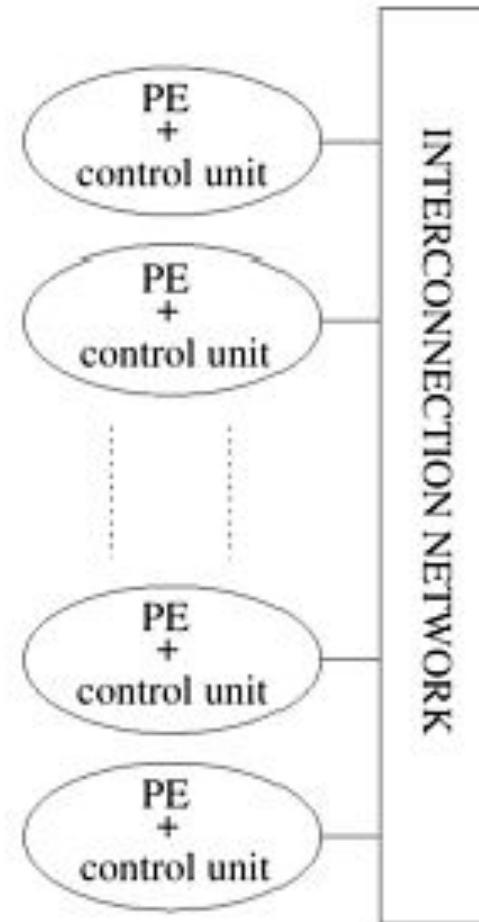
- various iterations of the loop are independent of each other; i.e.,
- `c[0] = a[0] + b[0]; c[1] = a[1] + b[1];`, etc., can all be executed independently of each other.
- Consequently, if there is a mechanism for executing the same instruction, in this case `add` on all the processors with appropriate data, we could execute this loop much faster.
- `Add` instruction is dispatched to all processors and executed concurrently by them.

# DICHOTOMY OF PARALLEL COMPUTING PLATFORMS

A typical SIMD architecture



A typical MIMD architecture



# DICHOTOMY OF PARALLEL COMPUTING PLATFORMS

- While the SIMD concept works well for structured computations on parallel data structures such as arrays, often it is necessary to selectively turn off operations on certain data items.
- For this reason, most SIMD programming paradigms allow for an "activity mask".
- This is a binary mask associated with each data item and operation that specifies whether it should participate in the operation or not
- Primitives such as `where (condition) then <stmt> <elsewhere stmt>` are used to support selective execution.
- Conditional execution can be detrimental to the performance of SIMD processors and therefore must be used with care.

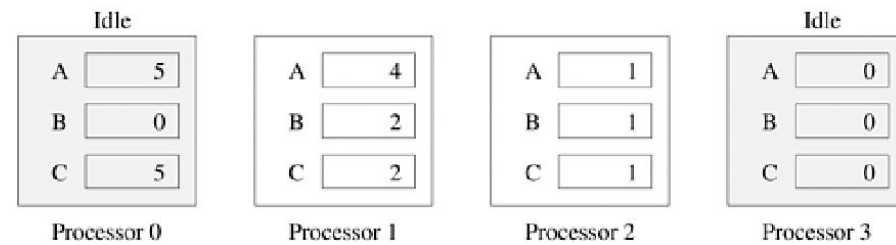
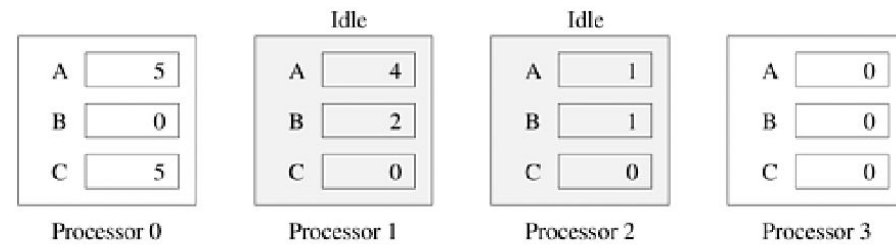
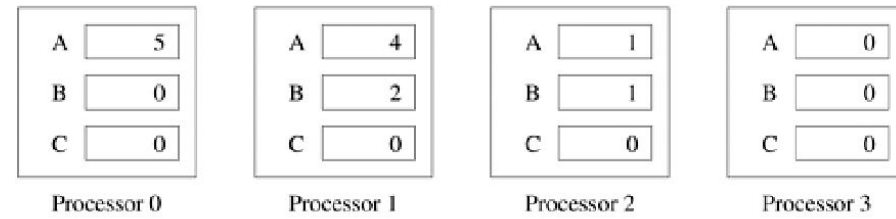
# SIMD VS MIMD

- SIMD computers require less hardware than MIMD computers because they have only one global control unit.
- Furthermore, SIMD computers require less memory because only one copy of the program needs to be stored.
- In contrast, MIMD computers store the program and operating system at each processor.
- However, the relative unpopularity of SIMD processors as general-purpose compute engines can be attributed to their specialized hardware architectures, economic factors, design constraints, product life-cycle, and application characteristics

# SIMD EXAMPLE

```
if (B == 0)
    C = A;
else
    C = A/B;
```

(a)



(b)

# SIMD EXAMPLE

- Consider the execution of a conditional statement previously.
- The conditional statement is executed in two steps.
- In the first step, all processors that have  $B$  equal to zero execute the instruction  $C = A$ .
- All other processors are idle. In the second step, the 'else' part of the instruction ( $C = A/B$ ) is executed.
- The processors that were active in the first step now become idle.
- **This illustrates one of the drawbacks of SIMD architectures.**

# COMMUNICA TION MODEL OF PARALLEL PLATFORMS

---

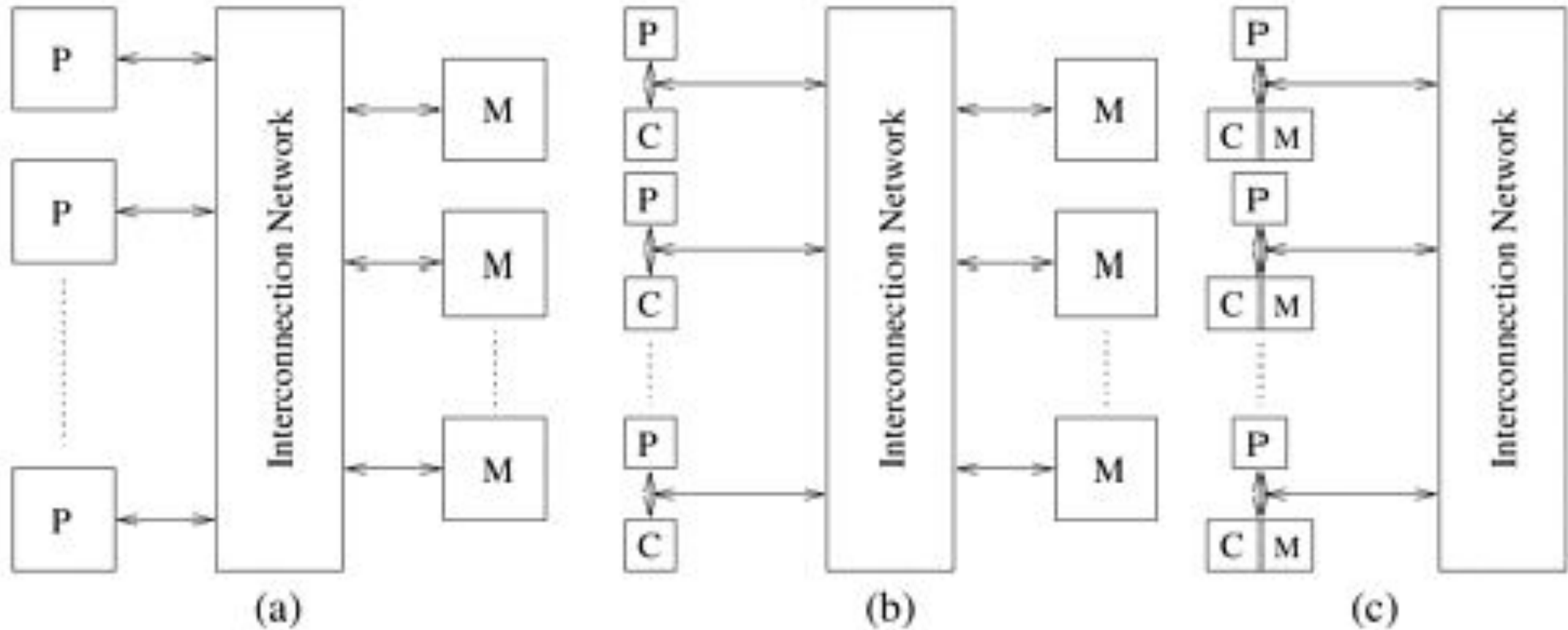


## SHARED-ADDRESS-SPACE PLATFORMS

- The "shared-address-space" view of a parallel platform supports a common data space that is accessible to all processors.
- Processors interact by modifying data objects stored in this shared address-space.
- Memory in shared-address-space platforms can be local (exclusive to a processor) or global (common to all processors).
- If the time taken by a processor to access any memory word in the system (global or local) is identical, the platform is classified as a **uniform memory access** (UMA) multicomputer.
- On the other hand, if the time taken to access certain memory words is longer than others, the platform is called a **nonuniform memory access** (NUMA) multicomputer.



# SHARED-ADDRESS-SPACE PLATFORMS

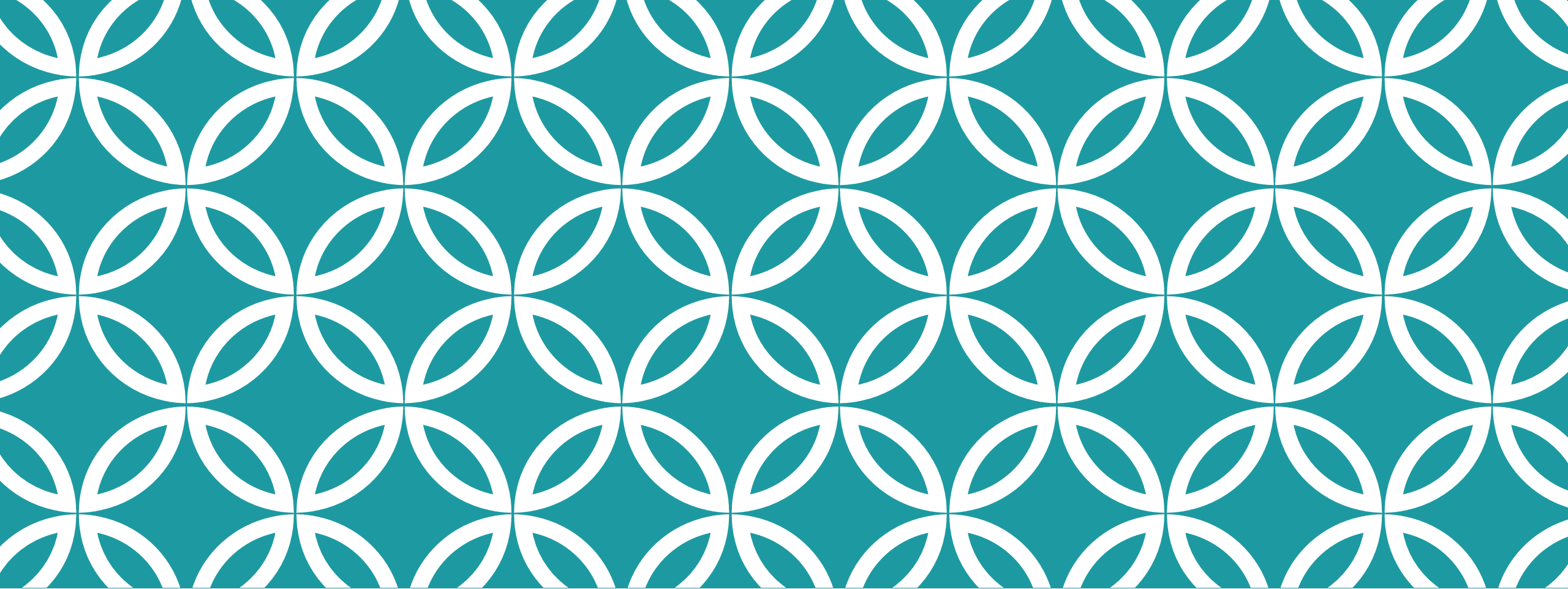


## SHARED-ADDRESS-SPACE PLATFORMS

- Figures 2.5(a) and (b) illustrate UMA platforms, whereas Figure 2.5(c) illustrates a NUMA platform.
- An interesting case is illustrated in Figure 2.5(b). Here, it is faster to access a memory word in cache than a location in memory.
- However, we still classify this as a UMA architecture. The reason for this is that all current microprocessors have cache hierarchies.
- Consequently, even a uniprocessor would not be termed UMA if cache access times are considered. For this reason, we define NUMA and UMA architectures only in terms of memory access times and not cache access times.
- The distinction between UMA and NUMA platforms is important.
- If accessing local memory is cheaper than accessing global memory, algorithms must build locality and structure data and computation accordingly.

## MESSAGE-PASSING PLATFORMS

- The logical machine view of a message-passing platform consists of  $p$  processing nodes, each with its own exclusive address space.
- Each of these processing nodes can either be single processors or a shared-address-space multiprocessor – a trend that is fast gaining momentum in modern message-passing parallel computers.
- Instances of such a view come naturally from clustered workstations and non-shared-address-space multicomputers.
- On such platforms, interactions between processes running on different nodes must be accomplished using messages, hence the name ***message passing***.
- This exchange of messages is used to transfer data, work, and to synchronize actions among the processes.
- In its most general form, message-passing paradigms support execution of a different program on each of the  $p$  nodes.



## PHYSICAL ORGANIZATION OF PARALLEL PLATFORMS

Section 2.4

# PHYSICAL ORGANIZATION OF PARALLEL PLATFORMS

- A natural extension of the serial model of computation (the Random Access Machine, or RAM) consists of  $p$  processors and a global memory of unbounded size that is uniformly accessible to all processors.
- All processors access the same address space.
- Processors share a common clock but may execute different instructions in each cycle.
- This ideal model is also referred to as a ***parallel random access machine (PRAM)***.
- Since PRAMs allow concurrent access to various memory locations, depending on how simultaneous memory accesses are handled, PRAMs can be divided into four subclasses.

# PHYSICAL ORGANIZATION OF PARALLEL PLATFORMS

- **Exclusive-read, exclusive-write (EREW) PRAM.** In this class, access to a memory location is exclusive. No concurrent read or write operations are allowed. This is the weakest PRAM model, affording minimum concurrency in memory access.
- **Concurrent-read, exclusive-write (CREW) PRAM.** In this class, multiple read accesses to a memory location are allowed. However, multiple write accesses to a memory location are serialized.
- **Exclusive-read, concurrent-write (ERCW) PRAM.** Multiple write accesses are allowed to a memory location, but multiple read accesses are serialized.
- **Concurrent-read, concurrent-write (CRCW) PRAM.** This class allows multiple read and write accesses to a common memory location. This is the most powerful PRAM model.

# PHYSICAL ORGANIZATION OF PARALLEL PLATFORMS

- Allowing concurrent read access does not create any semantic discrepancies in the program.
- However, concurrent write access to a memory location requires arbitration.
- Several protocols are used to resolve concurrent writes. The most frequently used protocols are as follows:
  - **Common**, in which the concurrent write is allowed if all the values that the processors are attempting to write are identical.
  - **Arbitrary**, in which an arbitrary processor is allowed to proceed with the write operation and the rest fail.
  - **Priority**, in which all processors are organized into a predefined prioritized list, and the processor with the highest priority succeeds and the rest fail.
  - **Sum**, in which the sum of all the quantities is written (the sum-based write conflict resolution model can be extended to any associative operator defined on the quantities being written).

## ARCHITECTURAL COMPLEXITY OF THE IDEAL MODEL

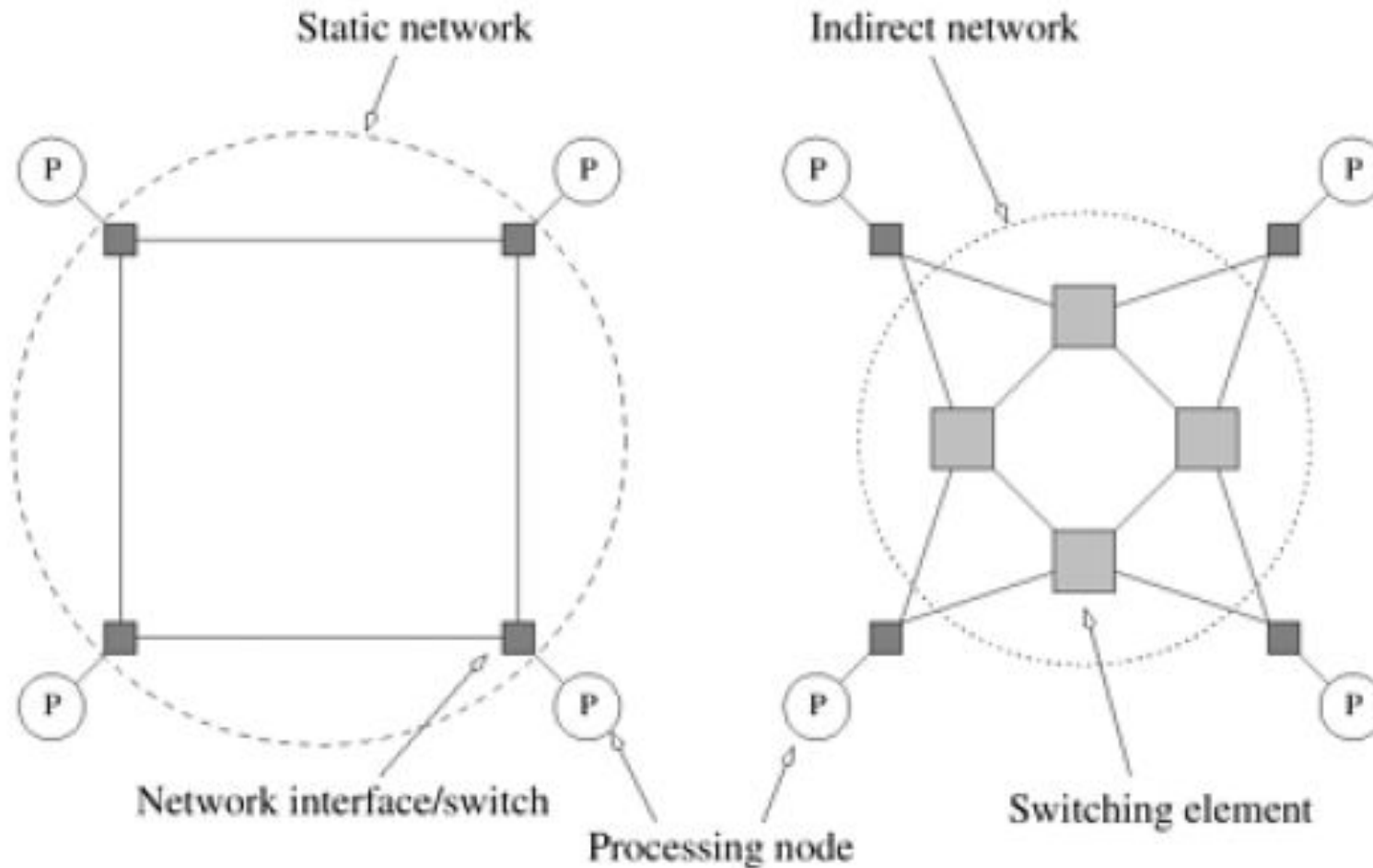
- Consider the implementation of an EREW PRAM as a shared-memory computer with  $p$  processors and a global memory of  $m$  words.
- The processors are connected to the memory through a set of switches.
- These switches determine the memory word being accessed by each processor.
- In an EREW PRAM, each of the  $p$  processors in the ensemble can access any of the memory words, provided that a word is not accessed by more than one processor simultaneously.
- To ensure such connectivity, the total number of switches must be  $\Theta(mp)$ .
- **For a reasonable memory size, constructing switching network of this complexity is very expensive.**
- Thus, PRAM models of computation are impossible to realize in practice.



## INTERCONNECTION NETWORKS FOR PARALLEL COMPUTERS

- Interconnection networks can be classified as ***static*** or ***dynamic***.
- Static networks consist of point-to-point communication links among processing nodes and are also referred to as ***direct*** networks.
- Dynamic networks, on the other hand, are built using switches and communication links.
- Communication links are connected to one another dynamically by the switches to establish paths among processing nodes and memory banks.
- Dynamic networks are also referred to as ***indirect*** networks

## INTERCONNECTION NETWORKS FOR PARALLEL COMPUTERS



## INTERCONNECTION NETWORKS FOR PARALLEL COMPUTERS

- A single switch in an interconnection network consists of a set of input ports and a set of output ports.
- Switches provide a range of functionality.
- The minimal functionality provided by a switch is a mapping from the input to the output ports.
- The total number of ports on a switch is also called the **degree** of the switch.
- Switches may also provide support for internal buffering (when the requested output port is busy), routing (to alleviate congestion on the network), and multicast (same output on multiple ports).
- The mapping from input to output ports can be provided using a variety of mechanisms based on physical crossbars, multi-ported memories, multiplexor-demultiplexors, and multiplexed buses.
- The cost of a switch is influenced by the cost of the mapping hardware, the peripheral hardware and packaging costs.

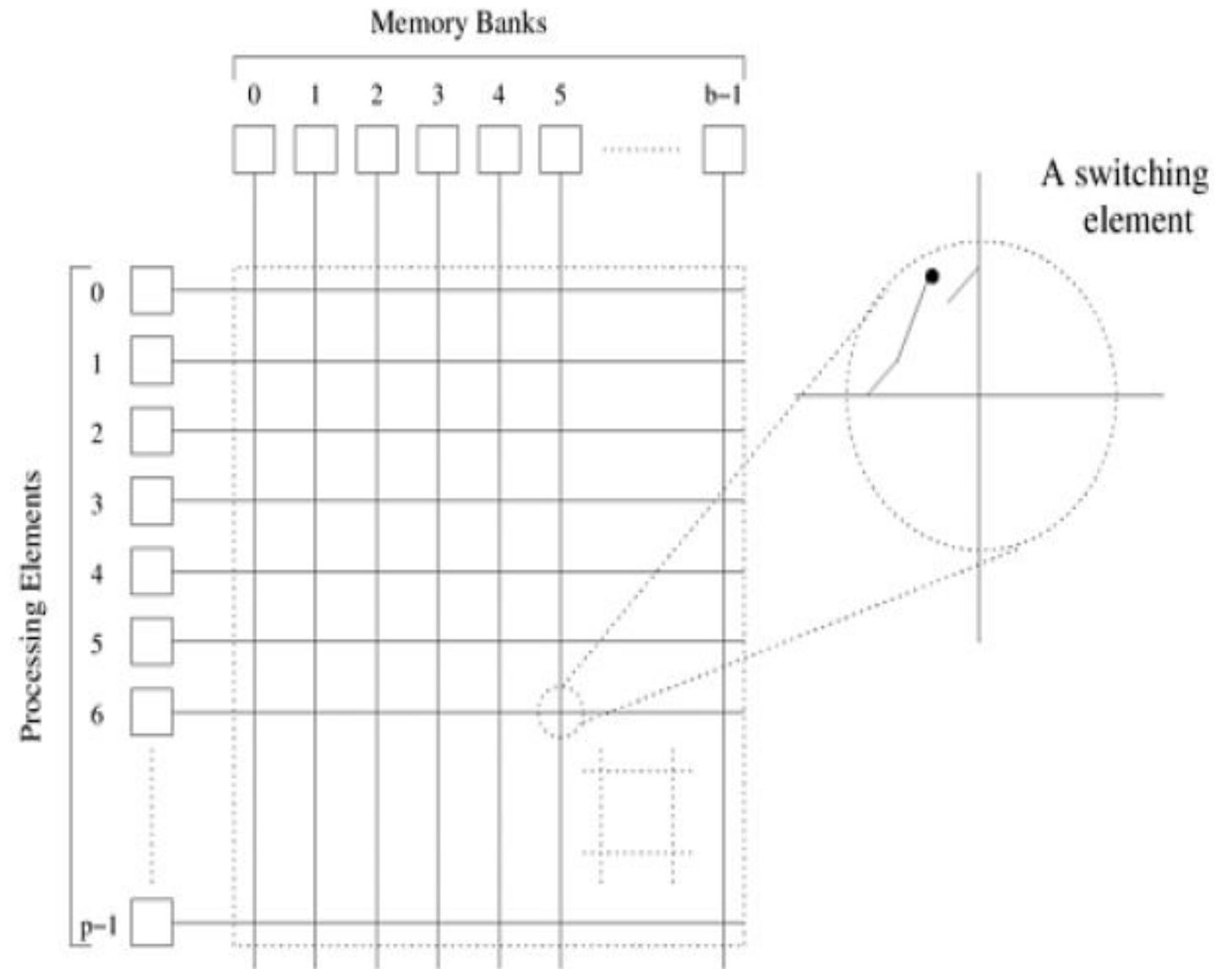
# NETWORK TOPOLOGIES-BUS-BASED NETWORKS

- A bus-based network is perhaps the simplest network consisting of a shared medium that is common to all the nodes.
- A bus has the desirable property that the cost of the network scales linearly as the number of nodes,  $p$ .
- This cost is typically associated with bus interfaces.
- Furthermore, the distance between any two nodes in the network is constant ( $O(1)$ ).
- Buses are also ideal for broadcasting information among nodes.
- Since the transmission medium is shared, there is little overhead associated with broadcast compared to point-to-point message transfer.
- However, the bounded bandwidth of a bus places limitations on the overall performance of the network as the number of nodes increases

# NETWORK TOPOLOGIES-CROSSBAR NETWORKS

- A simple way to connect  $p$  processors to  $b$  memory banks is to use a crossbar network.
- A crossbar network employs a grid of switches or switching nodes.
- The crossbar network is a non-blocking network in the sense that the connection of a processing node to a memory bank does not block the connection of any other processing nodes to other memory banks.

# NETWORK TOPOLOGIES-C ROSSBAR NETWORKS



# NETWORK TOPOLOGIES-CROSSBAR NETWORKS

- The total number of switching nodes required to implement such a network is  $\Theta(pb)$ .
- It is reasonable to assume that the number of memory banks  $b$  is at least  $p$ ; otherwise, at any given time, there will be some processing nodes that will be unable to access any memory banks.
- Therefore, as the value of  $p$  is increased, the complexity (component count) of the switching network grows as  $\Omega(p^2)$ .
- As the number of processing nodes becomes large, this switch complexity is difficult to realize at high data rates.
- Consequently, crossbar networks are not very scalable in terms of cost.

## NETWORK TOPOLOGIES-COMpletely-CONNECTED NETWORK

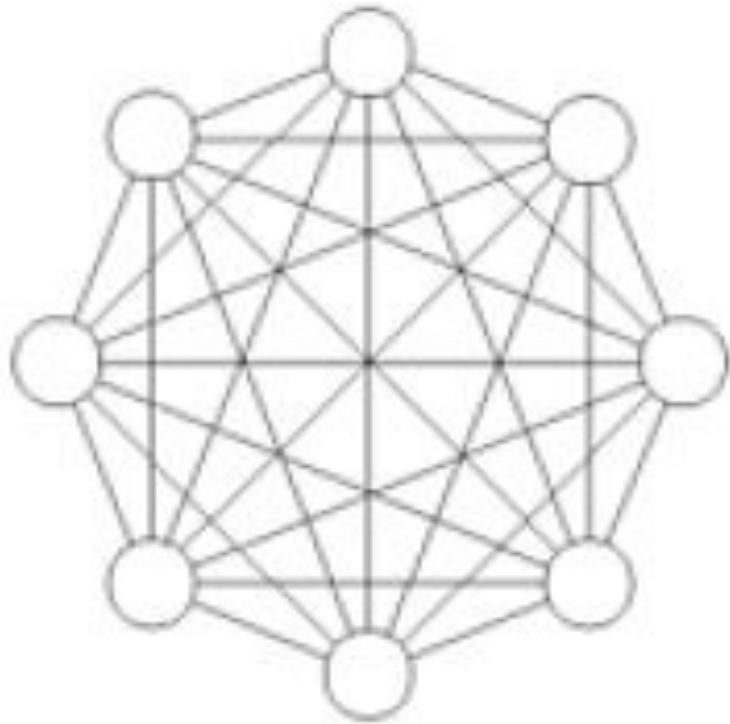
- In a ***completely-connected network***, each node has a direct communication link to every other node in the network.
- This network is ideal in the sense that a node can send a message to another node in a single step, since a communication link exists between them.
- Completely-connected networks are the static counterparts of crossbar switching networks, since in both networks, the communication between any input/output pair does not block communication between any other pair.



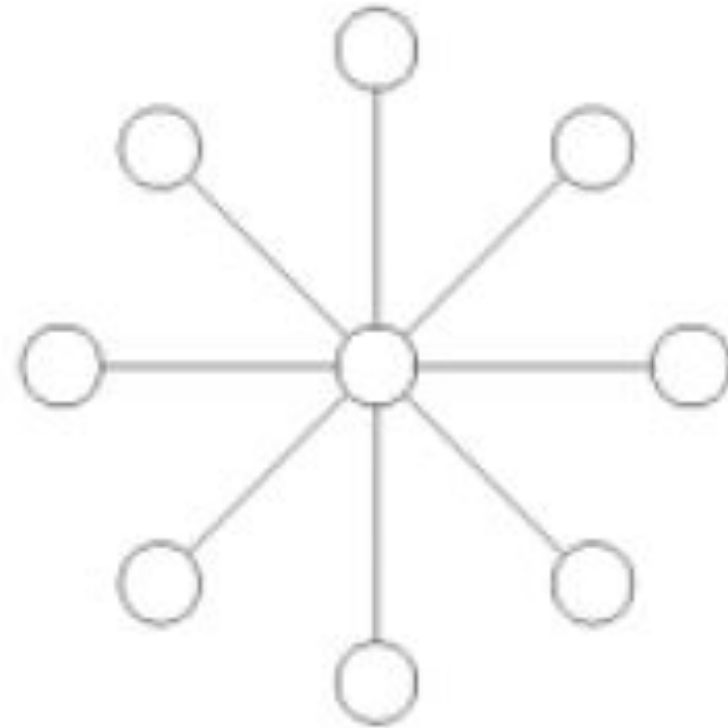
## NETWORK TOPOLOGIES-STAR-CONNECTED NETWORK

- In a ***star-connected network***, one processor acts as the central processor.
- Every other processor has a communication link connecting it to this processor.
- The star-connected network is similar to bus-based networks.
- Communication between any pair of processors is routed through the central processor, just as the shared bus forms the medium for all communication in a bus-based network.
- The central processor is the bottleneck in the star topology

## COMPLETELY CONNECTED NETWORK V/S STAR-CONNECTED NETWORK



(a)



(b)

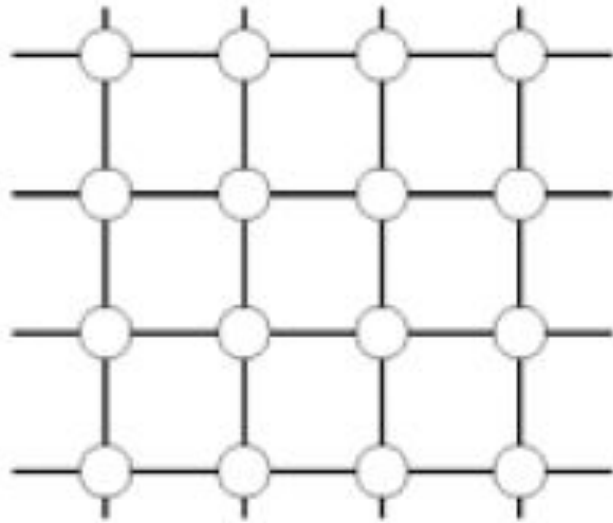
# LINEAR ARRAYS, MESHES, AND *K-D* MESHES

- Due to the large number of links in completely connected networks, sparser networks are typically used to build parallel computers.
- A family of such networks spans the space of linear arrays and hypercubes.
- A linear array is a static network in which each node (except the two nodes at the ends) has two neighbors, one each to its left and right.
- A simple extension of the linear array is the ring or a 1-D torus.
- The ring has a wraparound connection between the extremities of the linear array.
- In this case, each node has two neighbors.

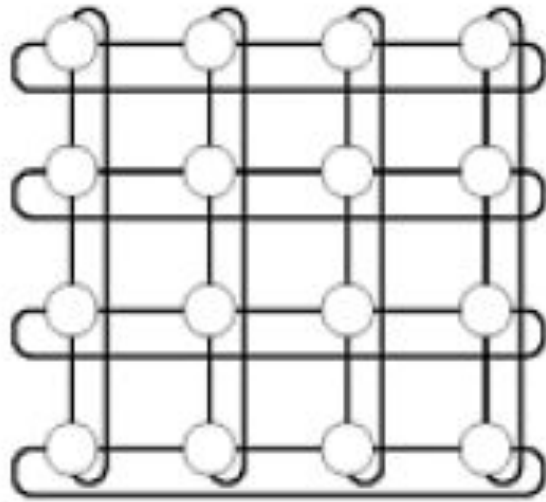
# LINEAR ARRAYS, MESHES, AND *K-D* MESHES

- A two-dimensional mesh is an extension of the linear array to two dimensions.
- Each dimension has nodes with a node identified by a two-tuple  $(i, j)$ .
- Every node (except those on the periphery) is connected to four other nodes whose indices differ in any dimension by one.
- A 2-D mesh has the property that it can be laid out in 2-D space, making it attractive from a wiring standpoint.
- Furthermore, a variety of regularly structured computations map very naturally to a 2-D mesh.
- For this reason, 2-D meshes were often used as interconnects in parallel machines.
- Two dimensional meshes can be augmented with wraparound links to form two dimensional tori. The three dimensional cube is a generalization of the 2-D mesh to three dimensions.
- Each node element in a 3-D cube, with the exception of those on the periphery, is connected to six other nodes, two along each of the three dimensions.
- A variety of physical simulations commonly executed on parallel computers (for example, 3-D weather modeling, structural modeling, etc.) can be mapped naturally to 3-D network topologies

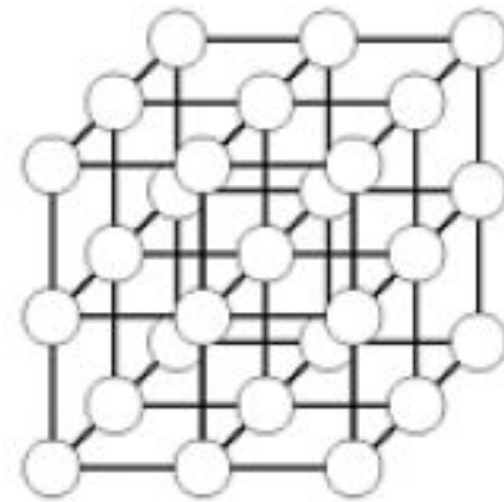
# LINEAR ARRAYS, MESHES, AND *K-D* MESHES



(a)



(b)

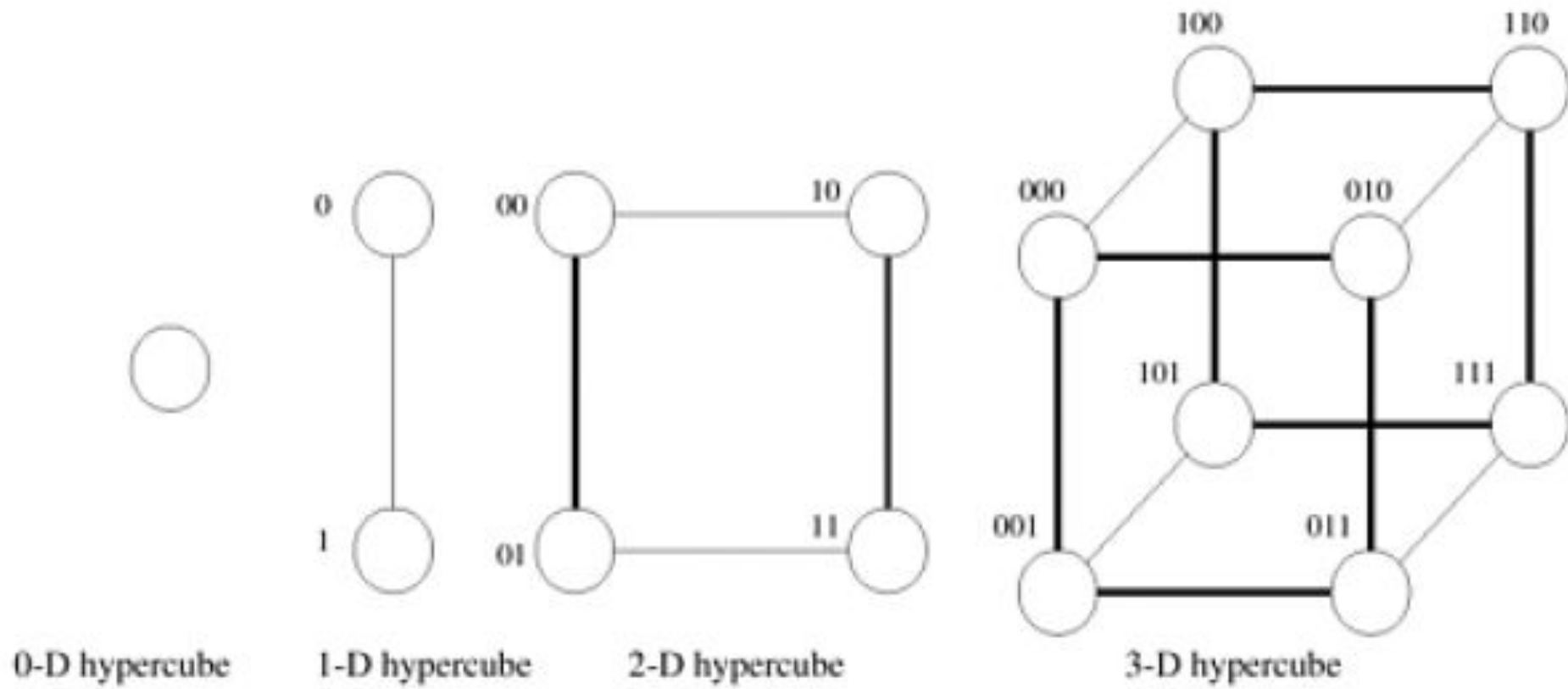


(c)

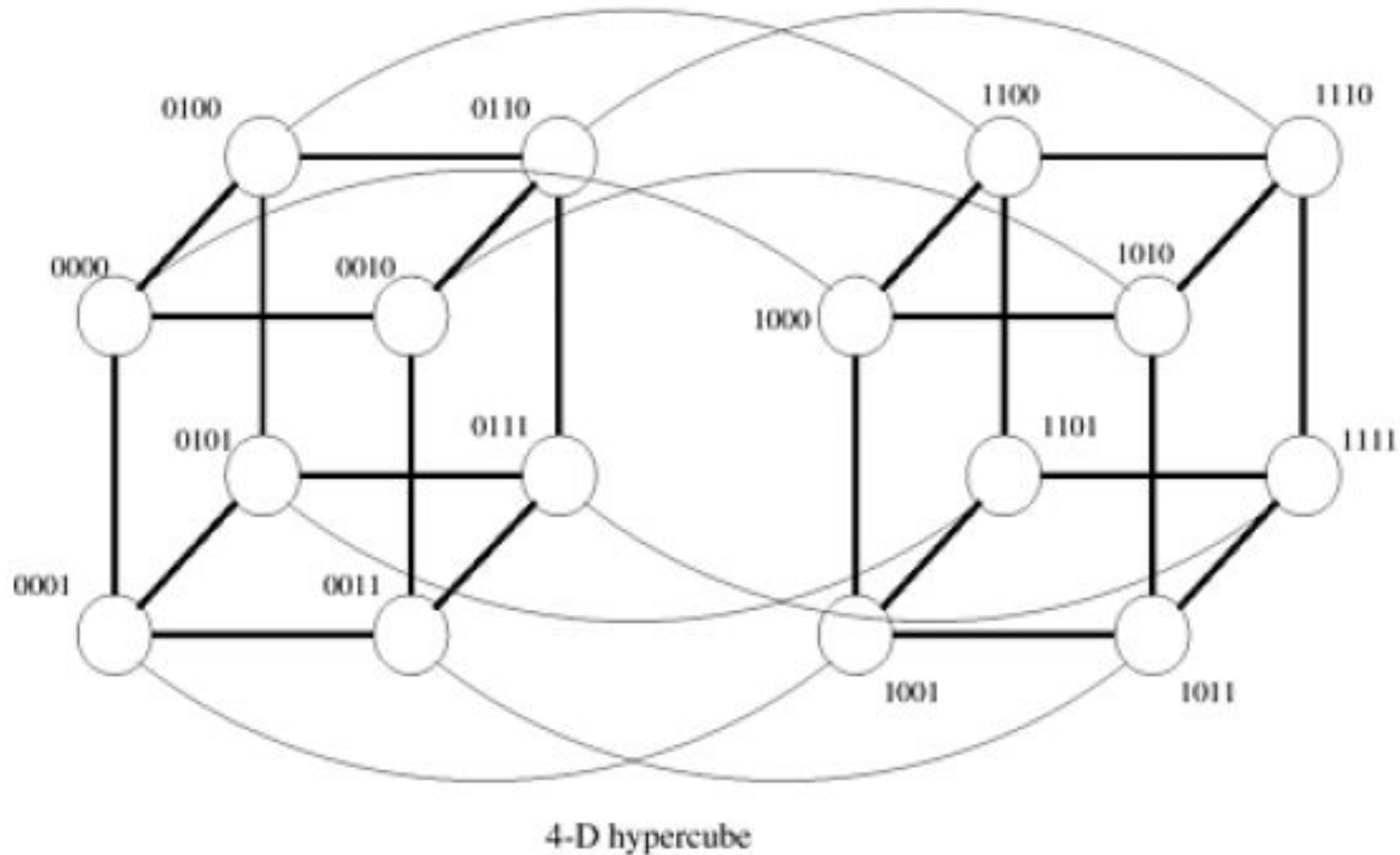
# LINEAR ARRAYS, MESHES, AND $K$ - $D$ MESHES

- The general class of  $k$ - $d$  meshes refers to the class of topologies consisting of  $d$  dimensions with  $k$  nodes along each dimension.
- Just as a linear array forms one extreme of the  $k$ - $d$  mesh family, the other extreme is formed by an interesting topology called the hypercube.
- The hypercube topology has two nodes along each dimension and  $\log p$  dimensions.
- A zero-dimensional hypercube consists of  $2^0$ , i.e., one node.
- A one-dimensional hypercube is constructed from two zero-dimensional hypercubes by connecting them.
- A two-dimensional hypercube of four nodes is constructed from two one-dimensional hypercubes by connecting corresponding nodes.
- In general a  $d$ -dimensional hypercube is constructed by connecting corresponding nodes of two  $(d - 1)$  dimensional hypercubes.

# LINEAR ARRAYS, MESHES, AND *K-D* MESHES



# LINEAR ARRAYS, MESHES, AND *K-D* MESHES





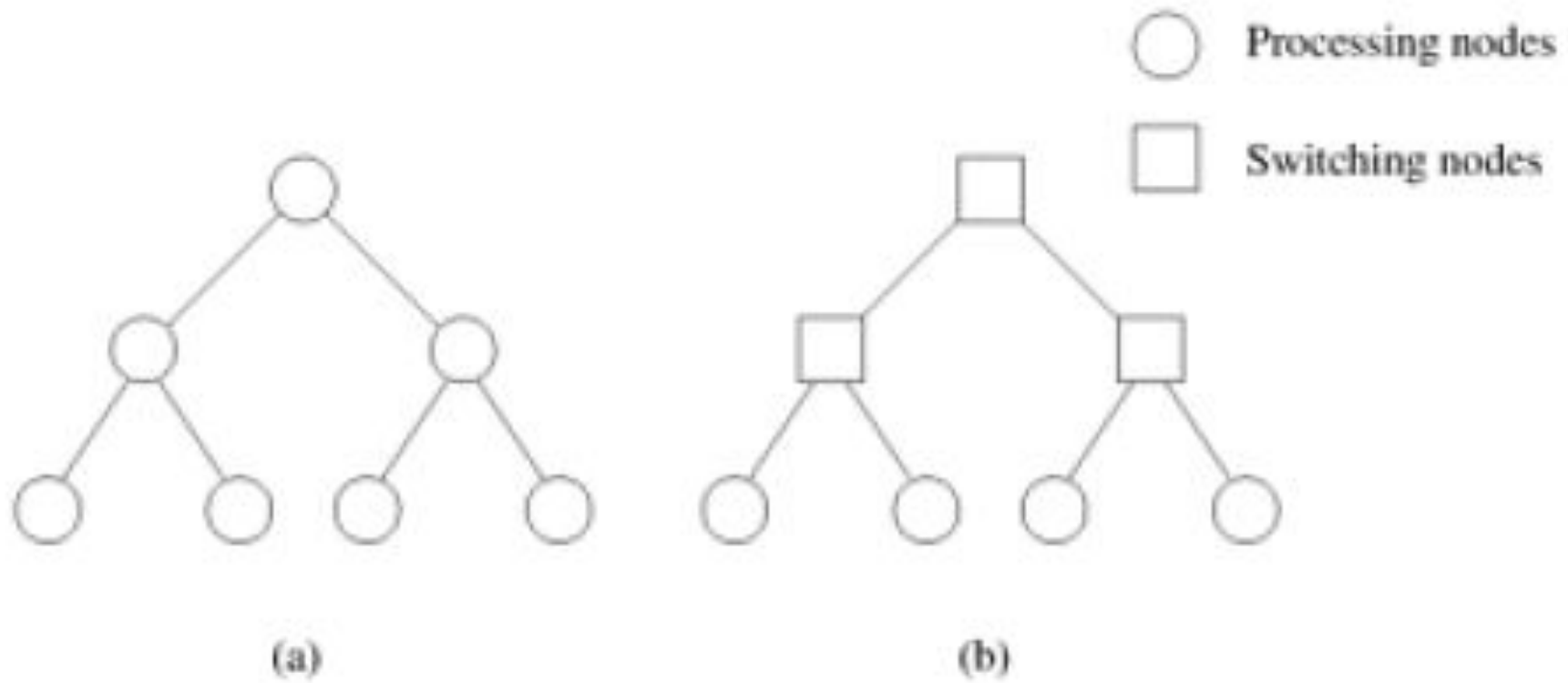
# LINEAR ARRAYS, MESHES, AND *K-D* MESHES

- It is useful to derive a numbering scheme for nodes in a hypercube.
- A simple numbering scheme can be derived from the construction of a hypercube.
- As illustrated in the figures before, if we have a numbering of two subcubes of  $p/2$  nodes, we can derive a numbering scheme for the cube of  $p$  nodes by prefixing the labels of one of the subcubes with a "0" and the labels of the other subcube with a "1".
- This numbering scheme has the useful property that the minimum distance between two nodes is given by the number of bits that are different in the two labels.
- For example, nodes labeled 0110 and 0101 are two links apart, since they differ at two bit positions.
- This property is useful for deriving a number of parallel algorithms for the hypercube architecture.

## TREE-BASED NETWORKS

- A ***tree network*** is one in which there is only one path between any pair of nodes.
- Both linear arrays and star-connected networks are special cases of tree networks.
- Static tree networks have a processing element at each node of the tree.
- Tree networks also have a dynamic counterpart.
- In a dynamic tree network, nodes at intermediate levels are switching nodes and the leaf nodes are processing elements.

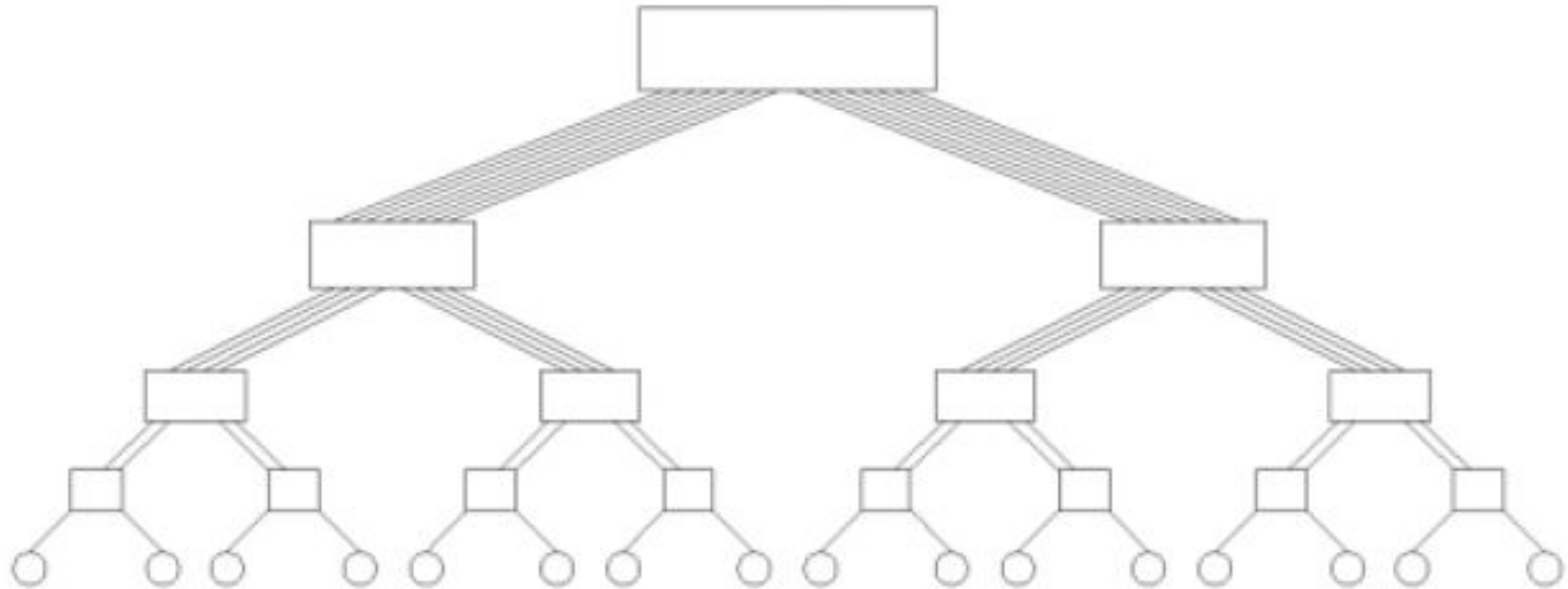
## TREE-BASED NETWORKS



## TREE-BASED NETWORKS

- To route a message in a tree, the source node sends the message up the tree until it reaches the node at the root of the smallest subtree containing both the source and destination nodes.
- Then the message is routed down the tree towards the destination node.
- Tree networks suffer from a communication bottleneck at higher levels of the tree.
- For example, when many nodes in the left subtree of a node communicate with nodes in the right subtree, the root node must handle all the messages.
- This problem can be alleviated in dynamic tree networks by increasing the number of communication links and switching nodes closer to the root.
- This network, is also called a ***fat tree***,

## TREE-BASED NETWORKS



## EVALUATING STATIC INTERCONNECTION NETWORKS

- **Diameter:** The *diameter* of a network is the maximum distance between any two processing nodes in the network. The distance between two processing nodes is defined as the shortest path (in terms of number of links) between them.
- **Connectivity:** The *connectivity* of a network is a measure of the multiplicity of paths between any two processing nodes. A network with high connectivity is desirable, because it lowers contention for communication resources. One measure of connectivity is the minimum number of arcs that must be removed from the network to break it into two disconnected networks. This is called the *arc connectivity* of the network.
- **Bisection Width and Bisection Bandwidth:** The *bisection width* of a network is defined as the minimum number of communication links that must be removed to partition the network into two equal halves.

## EVALUATING STATIC INTERCONNECTION NETWORKS

- The number of bits that can be communicated simultaneously over a link connecting two nodes is called the ***channel width***.
- Channel width is equal to the number of physical wires in each communication link.
- The peak rate at which a single physical wire can deliver bits is called the ***channel rate***.
- The peak rate at which data can be communicated between the ends of a communication link is called ***channel bandwidth***.
- Channel bandwidth is the product of channel rate and channel width.

**Channel bandwidth = channel rate \* channel width**

## EVALUATING STATIC INTERCONNECTION NETWORKS

- The **bisection bandwidth** of a network is defined as the minimum volume of communication allowed between any two halves of the network.
- It is the product of the bisection width and the channel bandwidth.
- Bisection bandwidth of a network is also sometimes referred to as **cross section bandwidth**.
- **Cost** Many criteria can be used to evaluate the cost of a network. One way of defining the cost of a network is in terms of the number of communication links or the number of wires required by the network.
- Linear arrays and trees use only  $p - 1$  links to connect  $p$  nodes.
- A  $d$ -dimensional wraparound mesh has  $dp$  links.
- A hypercube-connected network has  $(p \log p)/2$  links.



## EVALUATING STATIC INTERCONNECTION NETWORKS

- The bisection bandwidth of a network can also be used as a measure of its cost, as it provides a lower bound on the area in a two-dimensional packaging or the volume in a three-dimensional packaging.
- If the bisection width of a network is  $w$ , the lower bound on the area in a two-dimensional packaging is  $\Theta(w^2)$ , and the lower bound on the volume in a three-dimensional packaging is  $\Theta(w^{3/2})$ .
- According to this criterion, hypercubes and completely connected networks are more expensive than the other networks.

## EVALUATING STATIC INTERCONNECTION NETWORKS

| Network                       | Diameter                    | Bisection Width | Arc Connectivity | Cost (No. of links) |
|-------------------------------|-----------------------------|-----------------|------------------|---------------------|
| Completely-connected          | 1                           | $p^2/4$         | $p - 1$          | $p(p - 1)/2$        |
| Star                          | 2                           | 1               | 1                | $p - 1$             |
| Complete binary tree          | $2 \log((p + 1)/2)$         | 1               | 1                | $p - 1$             |
| Linear array                  | $p - 1$                     | 1               | 1                | $p - 1$             |
| 2-D mesh, no wraparound       | $2(\sqrt{p} - 1)$           | $\sqrt{p}$      | 2                | $2(p - \sqrt{p})$   |
| 2-D wraparound mesh           | $2\lfloor\sqrt{p}/2\rfloor$ | $2\sqrt{p}$     | 4                | $2p$                |
| Hypercube                     | $\log p$                    | $p/2$           | $\log p$         | $(p \log p)/2$      |
| Wraparound $k$ -ary $d$ -cube | $d\lfloor k/2\rfloor$       | $2k^{d-1}$      | $2d$             | $dp$                |

## EVALUATING DYNAMIC INTERCONNECTION NETWORKS

- A number of evaluation metrics for dynamic networks follow from the corresponding metrics for static networks.
- Since a message traversing a switch must pay an overhead, it is logical to think of each switch as a node in the network, in addition to the processing nodes.
- **The diameter of the network can now be defined as the maximum distance between any two nodes in the network.**
- This is indicative of the maximum delay that a message will encounter in being communicated between the selected pair of nodes.
- In reality, we would like the metric to be the maximum distance between any two processing nodes; however, for all networks of interest, this is equivalent to the maximum distance between any (processing or switching) pair of nodes.

## EVALUATING DYNAMIC INTERCONNECTION NETWORKS

- **The connectivity of a dynamic network can be defined in terms of node or edge connectivity.**
- The node connectivity is the minimum number of nodes that must fail (be removed from the network) to fragment the network into two parts.
- As before, we should consider only switching nodes (as opposed to all nodes).
- However, considering all nodes gives a good approximation to the multiplicity of paths in a dynamic network.
- The arc connectivity of the network can be similarly defined as the minimum number of edges that must fail (be removed from the network) to fragment the network into two unreachable parts.

## EVALUATING DYNAMIC INTERCONNECTION NETWORKS

- **The bisection width of a dynamic network must be defined more precisely than diameter and connectivity.**
- In the case of bisection width, we consider any possible partitioning of the  $p$  processing nodes into two equal parts.
- Note that this does not restrict the partitioning of the switching nodes.
- For each such partition, we select an induced partitioning of the switching nodes such that the number of edges crossing this partition is minimized.
- The minimum number of edges for any such partition is the bisection width of the dynamic network.
- Another intuitive way of thinking of bisection width is in terms of the minimum number of edges that must be removed from the network so as to partition the network into two halves with identical number of processing nodes.

## EVALUATING DYNAMIC INTERCONNECTION NETWORKS

| Network       | Diameter   | Bisection Width | Arc Connectivity | Cost (No. of links) |
|---------------|------------|-----------------|------------------|---------------------|
| Crossbar      | 1          | $p$             | 1                | $p^2$               |
| Omega Network | $\log p$   | $p/2$           | 2                | $p/2$               |
| Dynamic Tree  | $2 \log p$ | 1               | 2                | $p - 1$             |

## CACHE COHERENCE IN MULTIPROCESSOR SYSTEMS

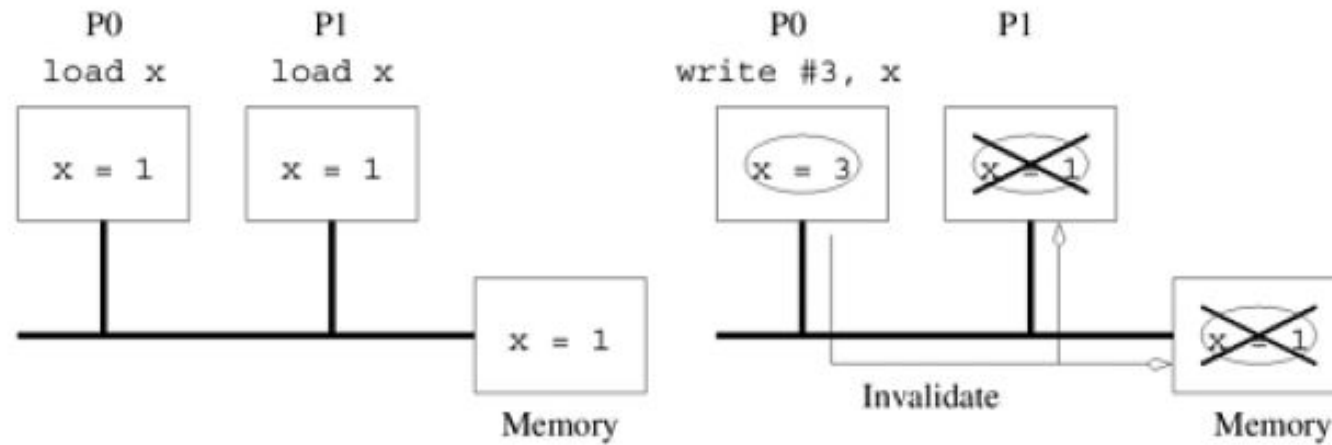
- While interconnection networks provide basic mechanisms for communicating messages (data), in the case of **shared-address-space** computers additional hardware is required to keep multiple copies of data consistent with each other.
- Specifically, if there exist two copies of the data (in different caches/memory elements), how do we ensure that different processors operate on these in a manner that follows predefined semantics?

## CACHE COHERENCE IN MULTIPROCESSOR SYSTEMS

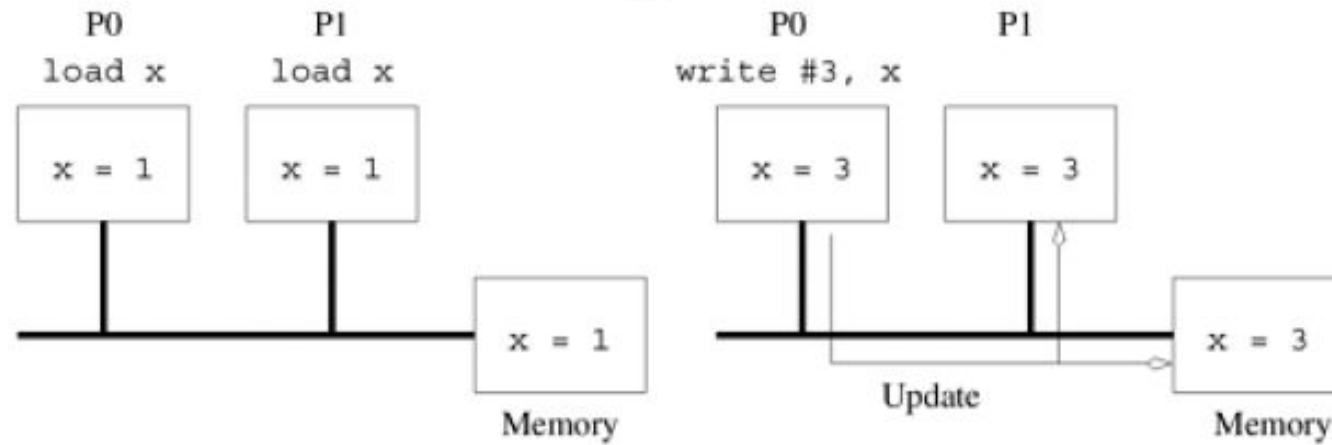
- The problem of keeping caches in multiprocessor systems coherent is significantly more complex than in uniprocessor systems.
- This is because in addition to multiple copies as in uniprocessor systems, there may also be multiple processors modifying these copies.
- Two processors  $P0$  and  $P1$  are connected over a shared bus to a globally accessible memory.
- Both processors load the same variable.
- There are now three copies of the variable.
- The coherence mechanism must now ensure that all operations performed on these copies are serializable (i.e., there exists some serial order of instruction execution that corresponds to the parallel schedule).
- When a processor changes the value of its copy of the variable, one of two things must happen: the other copies must be invalidated, or the other copies must be updated.
- Failing this, other processors may potentially work with incorrect (stale) values of the variable. These two protocols are referred to as **invalidate** and **update** protocols.



## CACHE COHERENCE IN MULTIPROCESSOR SYSTEMS



(a)



(b)

## CACHE COHERENCE IN MULTIPROCESSOR SYSTEMS

- In an update protocol, whenever a data item is written, all of its copies in the system are updated.
- For this reason, if a processor simply reads a data item once and never uses it, subsequent updates to this item at other processors cause excess overhead in terms of latency at source and bandwidth on the network.
- On the other hand, in this situation, an invalidate protocol invalidates the data item on the first update at a remote processor and subsequent updates need not be performed on this copy.

## CACHE COHERENCE IN MULTIPROCESSOR SYSTEMS

- Another important factor affecting the performance of these protocols is ***false sharing***.
- False sharing refers to the situation in which different processors update different parts of the same cache-line.
- Thus, although the updates are not performed on shared variables, the system does not detect this.
- In an invalidate protocol, when a processor updates its part of the cache-line, the other copies of this line are invalidated.
- When other processors try to update their parts of the cache-line, the line must actually be fetched from the remote processor.
- It is easy to see that false-sharing can cause a cache-line to be ping-ponged between various processors.
- In an update protocol, this situation is slightly better since all reads can be performed locally and the writes must be updated. This saves an invalidate operation that is otherwise wasted.

## CACHE COHERENCE IN MULTIPROCESSOR SYSTEMS

- The tradeoff between invalidate and update schemes is the classic tradeoff between communication overhead (updates) and idling (stalling in invalidates).
- Current generation cache coherent machines typically rely on invalidate protocols.
- The rest of our discussion of multiprocessor cache systems therefore assumes invalidate protocols.
- **Maintaining Coherence Using Invalidate Protocols** Multiple copies of a single data item are kept consistent by keeping track of the number of copies and the state of each of these copies.

# ASSIGNMENT 2

■ Study and present a report in the next lecture on the following topics.

1. How to maintain coherence using the invalidate protocol
2. Three state coherence protocol
3. Snoopy Cache systems and its performance
4. Directory based systems, its architecture and performance